循序渐进，学习开发 xv6-riscv
第 9 章 硬件定时器中断

汪辰

目录

01

硬件定时器中断

中断类型

- 本地（Local）中断
  - ✓ software interrupt
  - ✓ timer interrupt
- 全局（Global）中断
  - ✓ externel interrupt

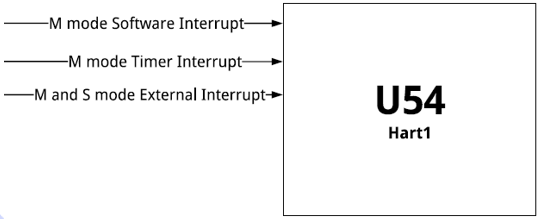


Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
1	0	*Reserved*
1	1	Supervisor software interrupt
1	2-4	*Reserved*
1	5	Supervisor timer interrupt
1	6-8	*Reserved*
1	9	Supervisor external interrupt
1	10-12	*Reserved*
1	13	Counter-overflow interrupt
1	14-15	*Reserved*
1	≥16	*Designated for platform use*

中断类型

- 本地（Local）中断
  - ✓ software interrupt
  - ✓ timer interrupt
- 全局（Global）中断
  - ✓ externel interrupt

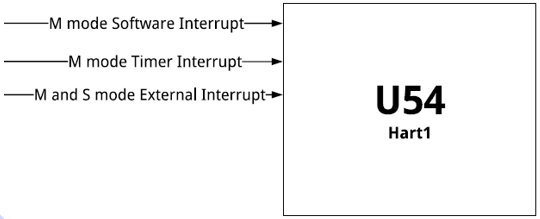


Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
1	0	*Reserved*
1	1	Supervisor software interrupt
1	2-4	*Reserved*
1	5	Supervisor timer interrupt
1	6-8	*Reserved*
1	9	Supervisor external interrupt
1	10-12	*Reserved*
1	13	Counter-overflow interrupt
1	14-15	*Reserved*
1	≥16	*Designated for platform use*

02

硬件定时器编程接口

Core Local INTerruptor

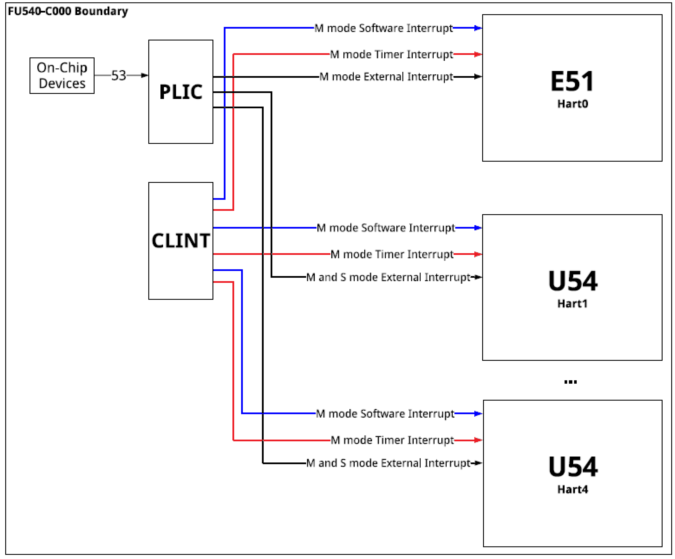


Figure 3: FU540-C000 Interrupt Architecture Block Diagram.

SiFive FU540-C000 Manual v1p0 Chapter 8.1

硬件定时器编程接口 – CSR

- 在 RISC-V 基础特权规范中仅定义了用于生成 Machine 模式定时器中断（CLINT）的硬件机制（基于 mtime 和 mtimecmp）。
- 编址采用内存映射（memory map）方式。具体编址采用 base + offset 的格式，且 base 由各个特定 platform 自己定义。
- 针对 QEMU-virt，其 CLINT 的设计参考了 SFIVE，base 为 0x2000000。
- 为提升性能、定义了 Sstc 扩展（基于 stimecmp ），使 Supervisor 模式能直接管理自己的定时器中断，大幅降低开销。

https://github.com/qemu/qemu/blob/master/hw/riscv/virt.

```
static const MemMapEntry virt_memmap[] = {
    [VIRT_DEBUG] =        {        0x0,         0x100 },
    [VIRT_MROM] =         {     0x1000,        0xf000 },
    [VIRT_TEST] =         {   0x100000,        0x1000 },
    [VIRT_RTC] =          {   0x101000,        0x1000 },
    [VIRT_CLINT] =        {  0x2000000,       0x10000 },
    [VIRT_ACLINT_SSWI] =  {  0x2F00000,        0x4000 },
    [VIRT_PCIE_PIO] =     {  0x3000000,       0x10000 },
    [VIRT_IOMMU_SYS] =    {  0x3010000,        0x1000 },
    [VIRT_PLATFORM_BUS] = {  0x4000000,     0x2000000 },
    [VIRT_PLIC] =         {  0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
    [VIRT_APLIC_M] =      {  0xc000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_APLIC_S] =      {  0xd000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_UART0] =        { 0x10000000,         0x100 },
    [VIRT_VIRTIO] =       { 0x10001000,        0x1000 },
    [VIRT_FW_CFG] =       { 0x10100000,          0x18 },
    [VIRT_FLASH] =        { 0x20000000,     0x4000000 },
    [VIRT_IMSIC_M] =      { 0x24000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_IMSIC_S] =      { 0x28000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_PCIE_ECAM] =    { 0x30000000,    0x10000000 },
    [VIRT_PCIE_MMIO] =    { 0x40000000,    0x40000000 },
    [VIRT_DRAM] =         { 0x80000000,           0x0 },
};
```

硬件定时器编程接口 - CSR

Figure 27. Machine time register (memory-mapped control register).

Figure 28. Machine time compare register (memory-mapped control register).

The RISC-V Instruction Set Manual Volume II，Chapter 3.2.1

- mtime 在 RV32 和 RV64 上都是 64-bit。系统必须保证该计数器的值始终按照一个固定的频率递增。如果计数溢出，mtime寄存器将会回绕。
- mtimecmp 也是 64-bit。当 mtime >= mtimecmp 时，CLINT 会产生一个 timer 中断（前提是需要保证全局中断打开并且 mie.MTIE 标志位置 1）。程序可以在 mtimecmp 中写入一个大于 mtime 的值清除中断。
- mtime 和 mtimecmp 的地址采用 memory-map 方式，由 platform 定义。mtime 全局只有一个，mtimecmp 是每个 hart 一个。

硬件定时器编程接口 - CSR

Figure 27. Machine time register (me

Figure 28. Machine time compare register

The RISC-V Instruction Set Manual V

Address	Width	Attr.	Description	Notes
0x200000	4B	RW	`msip` for hart 0	MSIP Registers (1 bit wide)
0x200004	4B	RW	`msip` for hart 1	
0x200008	4B	RW	`msip` for hart 2	
0x20000c	4B	RW	`msip` for hart 3	
0x200010	4B	RW	`msip` for hart 4	
0x204028			Reserved	
...				
0x20bff7				
0x204000	8B	RW	`mtimecmp` for hart 0	MTIMECMP Registers
0x204008	8B	RW	`mtimecmp` for hart 1	
0x204010	8B	RW	`mtimecmp` for hart 2	
0x204018	8B	RW	`mtimecmp` for hart 3	
0x204020	8B	RW	`mtimecmp` for hart 4	
0x204028			Reserved	
...				
0x20bff7				
0x20bff8	8B	RW	`mtime`	Timer Register
0x20c000			Reserved	

Table 36: CLINT Register Map

SiFive FU540-C000 Manual v1p0

- mtime 在 RV32 和 RV64 上都是 64-bit。的频率递增。如果计数溢出，mtime寄存
- mtimecmp 也是 64-bit。当 mtime >= m（前提是需要保证全局中断打开并且 mie中写入一个大于 mtime 的值清除中断。
- mtime 和 mtimecmp 的地址采用 memory-map 方式，由 platform 定义。mtime 全局只有一个，mtimecmp 是每个 hart 一个。

硬件定时器编程接口 - CSR

Figure 18. Counter-enable (`mcounteren`) register.

The RISC-V Instruction Set Manual Volume II，Chapter 3.1.11

- 特权规范为 mtime 定义了一个只读的影子（shadow）CSR 寄存器 time。
- 特权规范为 Machine 模式下定义了 Counter-Enable 寄存器 mcounteren，设置 TM 标志位为 1 后允许在 Supervisor 模式下读取 time 寄存器。

硬件定时器编程接口 - CSR

Figure 18. Counter-enable (`mcounteren`) register.

The RISC-V Instruction Set Manual Volume II，Chapter 3.1.11

- 特权规范为 mtime 定义了一个只读的影子（shadow）CSR 寄存器 time。
- 特权规范为 Machine 模式下定义了 Counter-Enable 寄存器 mcounteren，设置 TM 标志位为 1 后允许在 Supervisor 模式下读取 time 寄存器。

```
w_mcounteren(r_mcounteren() | 2);
```

硬件定时器编程接口 - CSR

- Sstc 扩展定义了 stimecmp（64-bit），为 Supervisor 模式提供了自己的定时器机制，stimecmp 不是 memory-map 的，是基于 CSR 方式定义的。
- Sstc 扩展使能后，为了触发 Supervisor 级别的 timer 中断，在已经使能了 delegation 且全局中断打开的前提下，我们还需要将 mie.STIE 标志位和 sie.STIE 标志位都置 1。以上条件满足后，当 time >= stimecmp 时，会触发 Supervisor 模式下收到一个 timer 中断；程序可以在 stimecmp 中写入一个大于 time 的值清除中断。
- 特权规范在 Machine 模式下定义了 Environment Configuration 寄存器 menvcfg，通过设置其 STCE 标志位为 1 可以使能 Sstc 扩展。

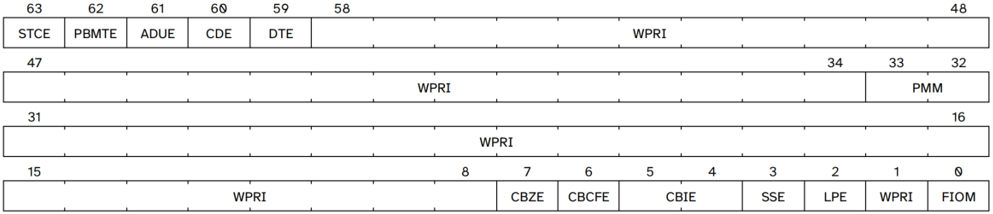


Figure 25. Machine environment configuration (menvcfg) register.

The RISC-V Instruction Set Manual Volume II，Chapter 3.1.18

硬件定时器编程接口 - CSR

- Sstc 扩展定义了 stimecmp（64-bit），为 Supervisor 模式提供了自己的定时器机制，stimecmp 不是 memory-map 的，是基于 CSR 方式定义的。
- Sstc 扩展使能后，为了触发 Supervisor 级别的 timer 中断，在已经使能了 delegation 且全局中断打开的前提下，我们还需要将 mie.STIE 标志位和 sie.STIE 标志位都置 1。以上条件满足后，当 time >= stimecmp 时，会触发 Supervisor 模式下收到一个 timer 中断；程序可以在 stimecmp 中写入一个大于 time 的值清除中断。
- 特权规范在 Machine 模式下定义了 Environment Configuration 寄存器 menvcfg，通过设置其 STCE 标志位为 1 可以使能 Sstc 扩展。

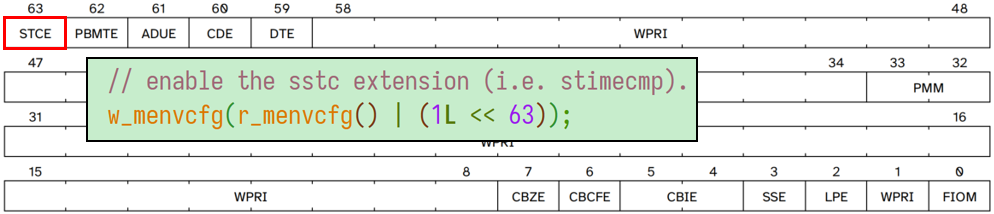


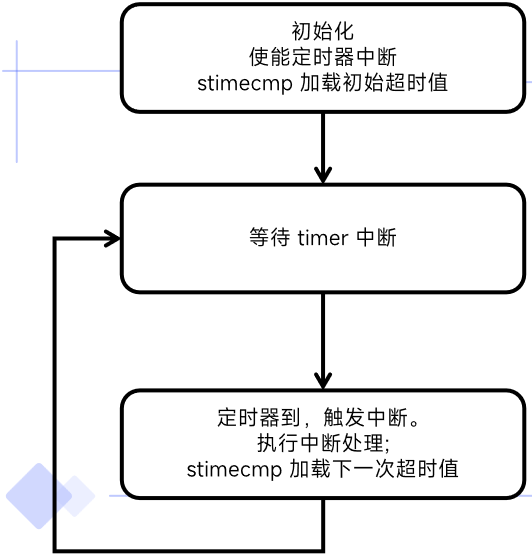
Figure 25. Machine environment configuration (menvcfg) register.

The RISC-V Instruction Set Manual Volume II，Chapter 3.1.18

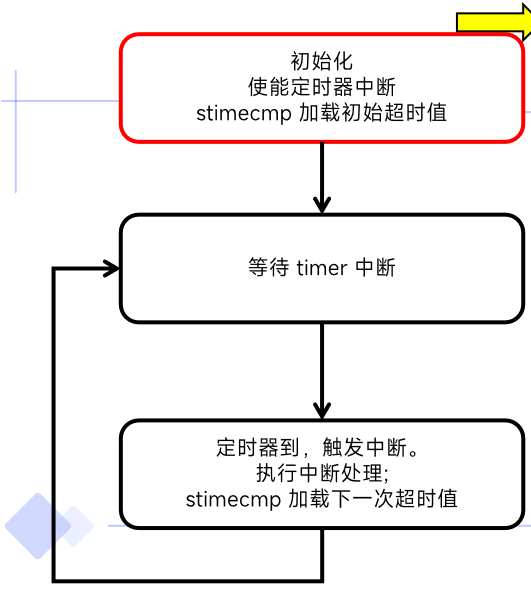
03

硬件定时器的实现

硬件定时器的实现

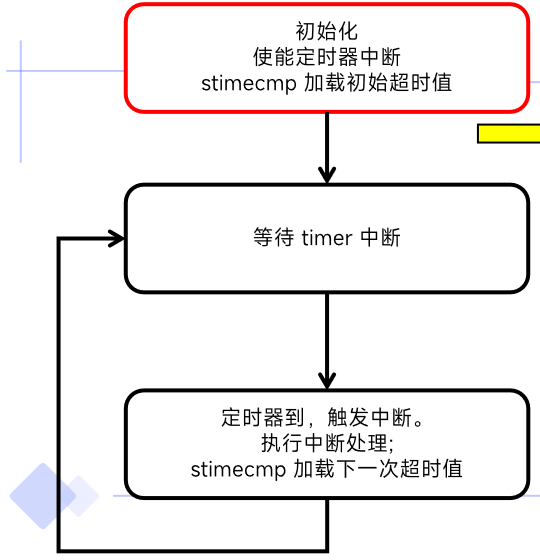

初始化
使能定时器中断
stimecmp 加载初始超时值
等待 timer 中断
定时器到，触发中断。
执行中断处理;
stimecmp 加载下一次超时值

硬件定时器的实现



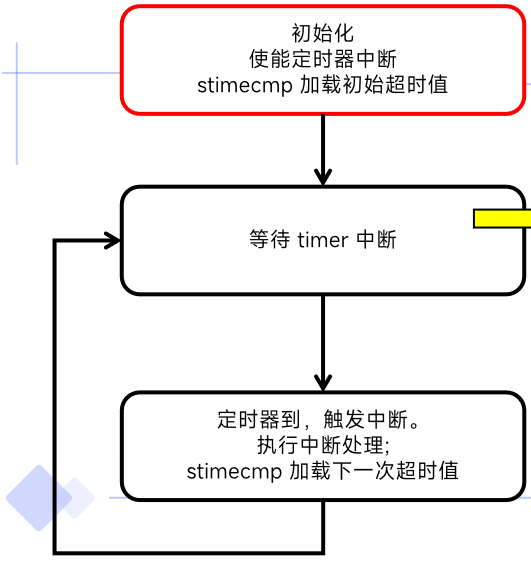

```c
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm volatile("mret");
}
```

硬件定时器的实现




```
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm volatile("mret");
}
```

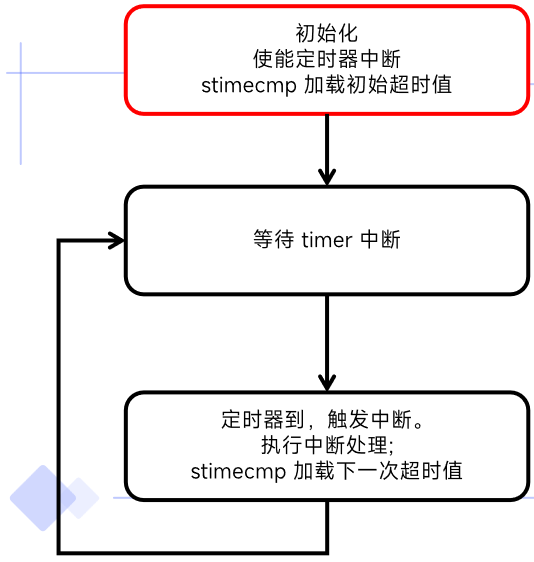
硬件定时器的实现




```c
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm
}
```

```c
void timerinit()
{
  // enable supervisor-mode timer interrupts.
  w_mie(r_mie() | MIE_STIE);
  // enable the sstc extension (i.e. stimecmp).
  w_menvcfg(r_menvcfg() | (1L << 63));
  // allow supervisor to use stimecmp and time.
  w_mcounteren(r_mcounteren() | 2);
  // ask for the very first timer interrupt.
  w_stimecmp(r_time() + 1000000);
}
```

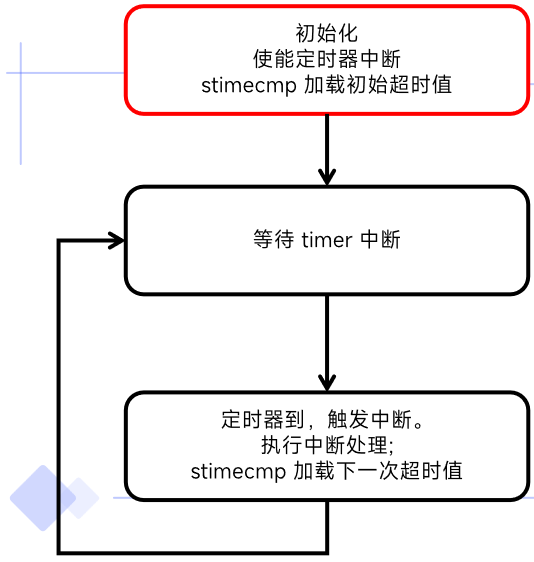
硬件定时器的实现




```c
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm
}
```

```c
void timerinit()
{
  // enable supervisor-mode timer interrupts.
  w_mie(r_mie() | MIE_STIE);
  // enable the sstc extension (i.e. stimecmp).
  w_menvcfg(r_menvcfg() | (1L << 63));
  // allow supervisor to use stimecmp and time.
  w_mcounteren(r_mcounteren() | 2);
  // ask for the very first timer interrupt.
  w_stimecmp(r_time() + 1000000);
}
```

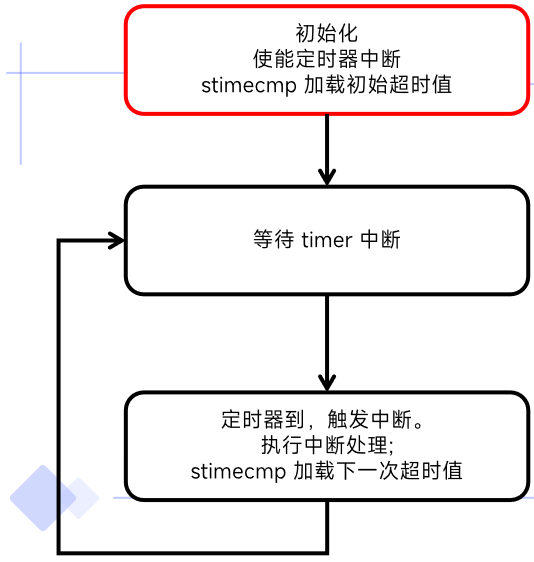
硬件定时器的实现




```c
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm
}
```

```c
void timerinit()
{
  // enable supervisor-mode timer interrupts.
  w_mie(r_mie() | MIE_STIE);
  // enable the sstc extension (i.e. stimecmp).
  w_menvcfg(r_menvcfg() | (1L << 63));
  // allow supervisor to use stimecmp and time.
  w_mcounteren(r_mcounteren() | 2);
  // ask for the very first timer interrupt.
  w_stimecmp(r_time() + 1000000);
}
```

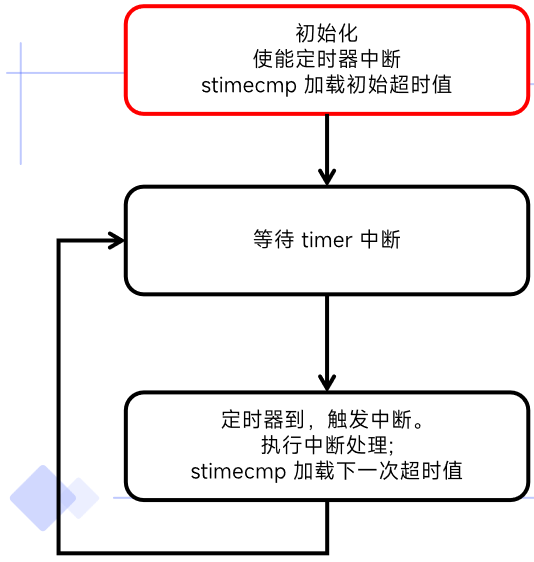
硬件定时器的实现




```c
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm
}
```

```c
void timerinit()
{
  // enable supervisor-mode timer interrupts.
  w_mie(r_mie() | MIE_STIE);
  // enable the sstc extension (i.e. stimecmp).
  w_menvcfg(r_menvcfg() | (1L << 63));
  // allow supervisor to use stimecmp and time.
  w_mcounteren(r_mcounteren() | 2);
  // ask for the very first timer interrupt.
  w_stimecmp(r_time() + 1000000);
}
```

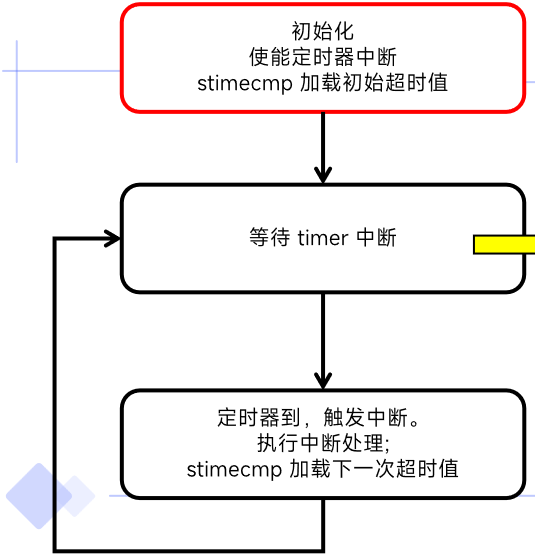
硬件定时器的实现




```
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm
}
```

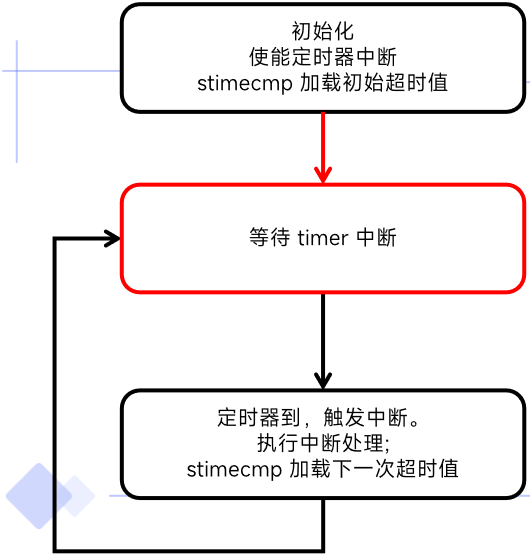
```
void timerinit()
{
  // enable supervisor-mode timer interrupts.
  w_mie(r_mie() | MIE_STIE);
  // enable the sstc extension (i.e. stimecmp).
  w_menvcfg(r_menvcfg() | (1L << 63));
  // allow supervisor to use stimecmp and time.
  w_mcounteren(r_mcounteren() | 2);
  // ask for the very first timer interrupt.
  w_stimecmp(r_time() + 1000000);
}
```

硬件定时器的实现



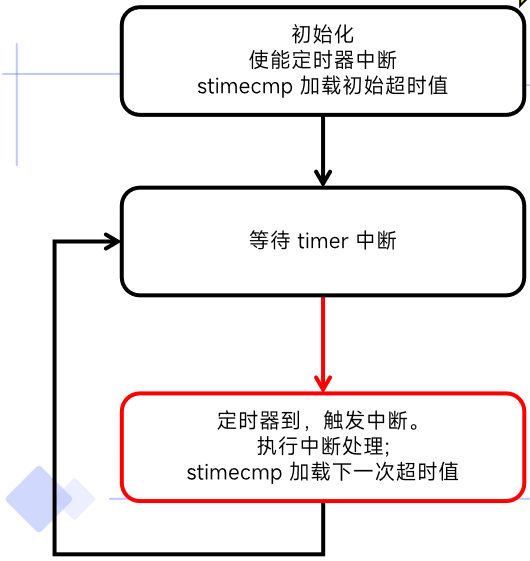

```
void start()
{
  // ......
  w_medeleg(0xffff);
  w_mideleg(0xffff);
  w_sie(r_sie() | SIE_SEIE | SIE_STIE);
  // ......
  // ask for clock interrupts.
  timerinit();
  asm volatile("mret");
}
```

硬件定时器的实现



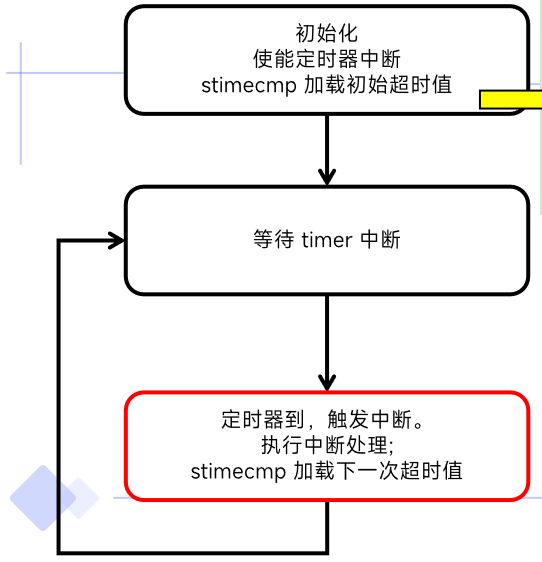

```
xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
```

硬件定时器的实现




```c
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which_dev == 2 && myproc() != 0)
    printf("--> timer interrupt\n");
}
```

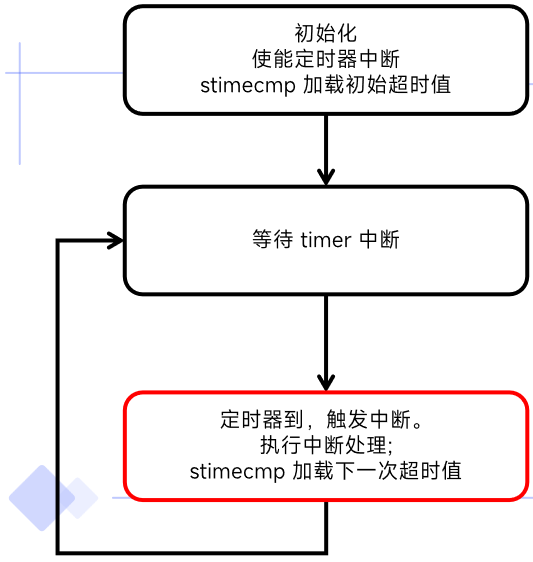
硬件定时器的实现




```c
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which
    printf
}
```

```c
int devintr()
{
  uint64 scause = r_scause();
  if(scause == 0x8000000000000009L){
    // ......
  } else if(scause == 0x8000000000000005L){
    // timer interrupt.
    clockintr();
    return 2;
  } else {
    return 0;
  }
}
```

硬件定时器的实现

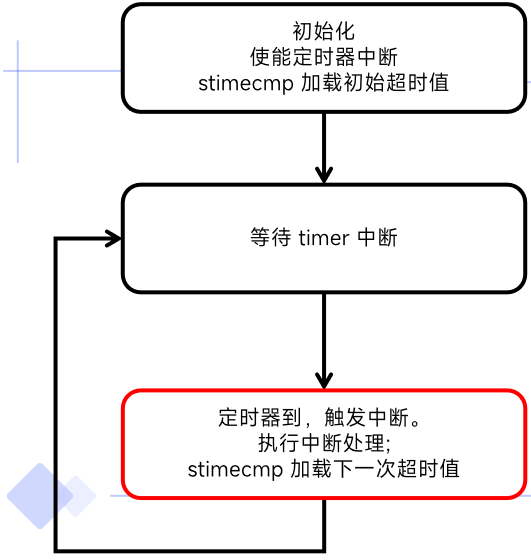



```
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which
    printf
}
```

```
int devintr()
{
  uint64 scause = r_scause();
  if(scause == 0x8000000000000009L){
    // ......
  } else if(scause == 0x8000000000000005L){
    // timer interrupt.
    clockintr();
    return
  } else {
    return
  }
}
```

```
void clockintr()
{
  w_stimecmp(r_time() + 1000000);
}
```

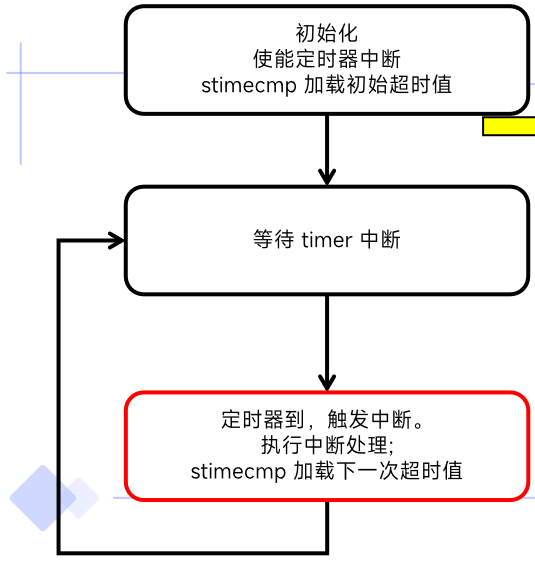
硬件定时器的实现




```
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which
    printf
}
```

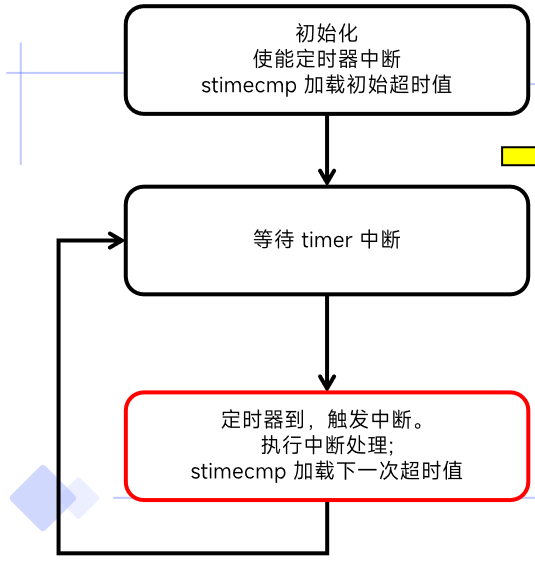
```
int devintr()
{
  uint64 scause = r_scause();
  if(scause == 0x8000000000000009L){
    // ......
  } else if(scause == 0x8000000000000005L){
    // timer interrupt.
    clockintr();
    return 2;
  } else {
    return 0;
  }
}
```

硬件定时器的实现



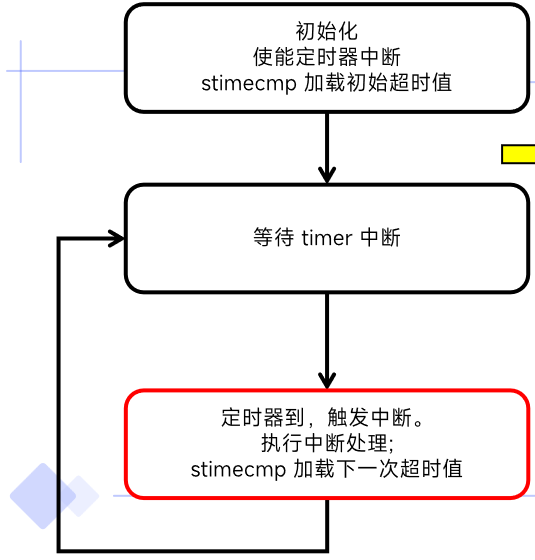

```c
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which_dev == 2 && myproc() != 0)
    printf("--> timer interrupt\n");
}
```

硬件定时器的实现




```c
void kerneltrap()
{
  int which_dev = 0;
  // ......
  which_dev = devintr();
  if(which_dev == 2 && myproc() != 0)
    printf("--> timer interrupt\n");
}
```

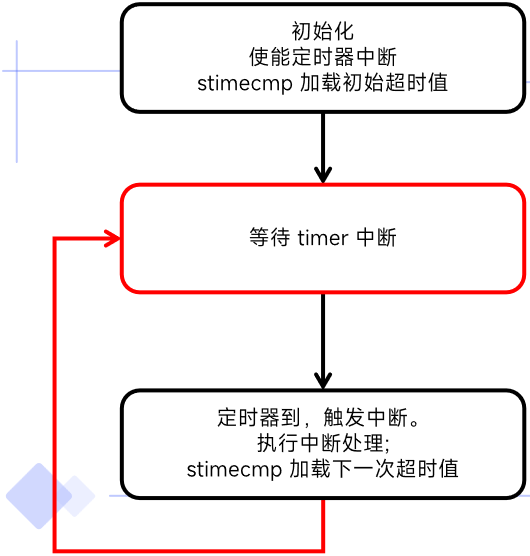
硬件定时器的实现




```
void kerneltrap()
{
  int w
  // ..
  which
  if(wh
    pri
}
```

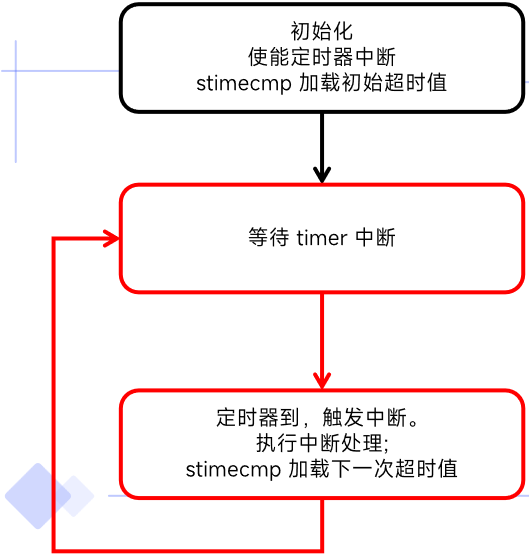
```
xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
--> timer interrupt
```

硬件定时器的实现



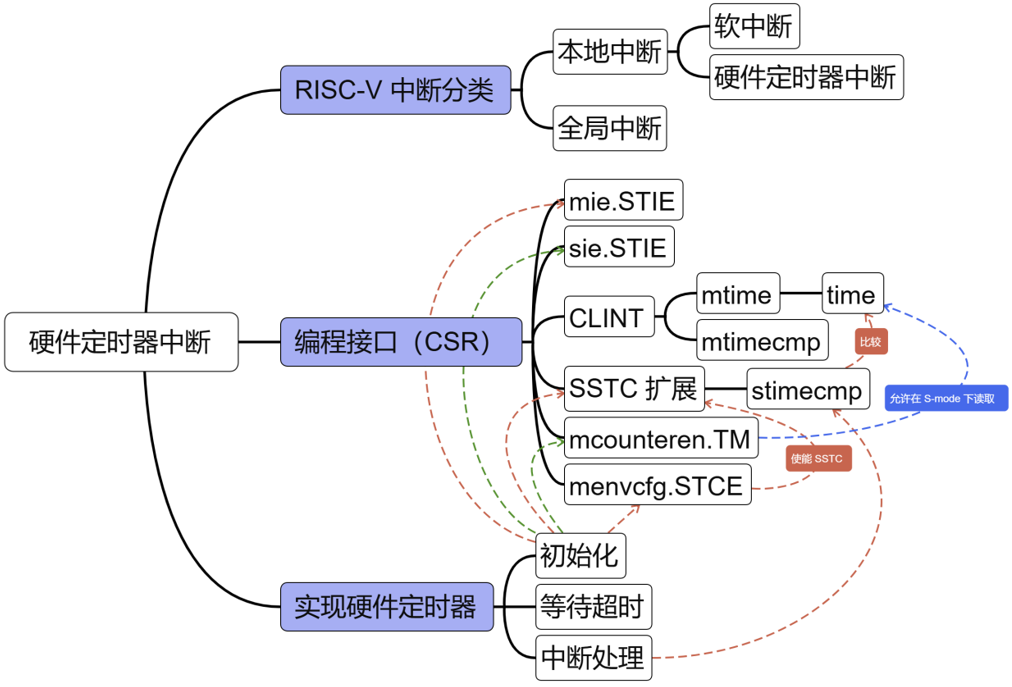

```
xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
--> timer interrupt
Task 0: Running...
```

硬件定时器的实现




```
xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
--> timer interrupt
Task 0: Running...
--> timer interrupt
Task 1: Running...
--> timer interrupt
Task 0: Running...
--> timer interrupt
Task 1: Running...
--> timer interrupt
Task 0: Running...
--> timer interrupt
Task 1: Running...
```

本章总结


硬件定时器中断
RISC-V 中断分类
本地中断
软中断
硬件定时器中断
全局中断
编程接口（CSR）
mie.STIE
sie.STIE
CLINT
mtime
time
mtimecmp
比较
SSTC 扩展
stimecmp
允许在 S-mode 下读取
mcounteren.TM
使能 SSTC
menvcfg.STCE
实现硬件定时器
初始化
等待超时
中断处理

谢谢